WebClient Computing

Javascript Basics

Department of Computer Science

Kyungyong Lee

Announcements

- Project due May 2nd (Monday)
- Project interview May 4th (Wednesday) 14:00 ~ 18:00
  - If you cannot make at the time window, send me a private message - @leeky
  - Interview slot booking - https://docs.google.com/spreadsheets/d/1eGbSY4iK8iJrUu2chUmHjyZ97tyox_5BhcxAy-PuqZY/edit?usp=sharing
- Homework-3 is out today
  - All the homework-3 related questions to #hw3-questions channel

About Array Sort

- By default, Array sort is based on the string in the ascending order

```
> [77,88,99,100].sort()
<· [100, 77, 88, 99]
```

```
> 123=="123";
<· true
```

Array Sort By Numbers

- array_sort.js
- We can set a comparator for array
- Array.sort(CompareFunction)

```
> function ascendingNumber(a,b) {
    return a - b;
  }
<· undefined
> arr1 = [88, 77, 100, 99];
<· [88, 77, 100, 99]
> console.log(arr1.sort(ascendingNumber));
  [77, 88, 99, 100]
```

More About Compare Function

```
function compare(a, b) {
        // return negative: a should be prior to b (ex. a=10,b=20 and a=20,b=10)
        // return positive: b should be prior to a
        // return 0: two values are the same
}
```

- How to sort in the descending order for the number?
- String order in the descending order, you can use reverse()

Conditional Statement

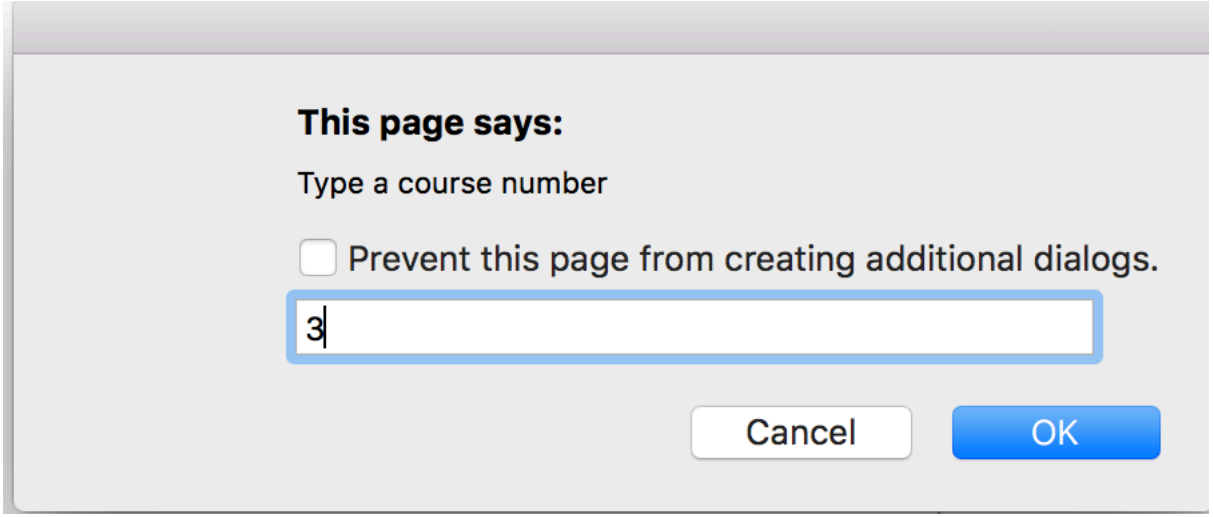
- if(boolean)
  - Performs command if the boolean is true
- If(boolean)-else
  - Performs if command if boolean is true. Otherwise, run else commands
- If(boolean-1)-else if(boolean-2)
  - Only if boolean-1 is false and boolean-2 is true, else if command is executed
- switch (condition) – case: break;– default
  - The condition in the switch is evaluated with case to select which command to execute
  - Missing break executes all the case statement until it meets break or the end

Conditional Statement Example (switch)

- switch_statement.js

```js
var input = parseInt(prompt("Type a course number"))

switch(input) {
  case 1:
    console.log("WebClient computing");
    break;
  case 2:
    console.log("WebServer computing");
    break;
  case 3:
    console.log("Data Structure");
    break;
  default:
    console.log("No course");
    break;
}
```

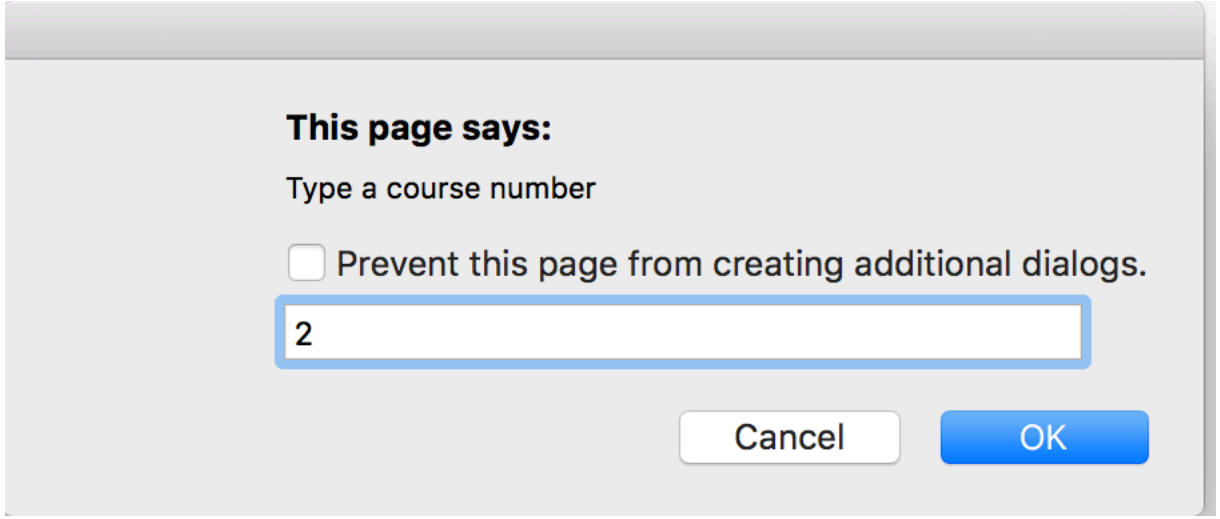



```
Data Structure
```

Switch Statement Missing break

```
var input = parseInt(prompt("Type a course number"))

switch(input) {
  case 1:
    console.log("WebClient computing");
  case 2:
    console.log("WebServer computing");
  case 3:
    console.log("Data Structure");
  default:
    console.log("No course");
}
```




```
WebServer computing
Data Structure
No course
```

Omitting break in Switch Intentionally

- switch_no_break.js

```
var today = "Friday";

switch(today) {
  case "Monday":
  case "Tuesday":
  case "Wednesday":
  case "Thursday":
  case "Friday":
    console.log("Today is weekday");
    break;
  case "Saturday":
  case "Sunday":
    console.log("Today is weekend");
    break;
}
```

Loop in Javascript

- while(condition)
  - Execute commands in the while statement until the condition is true
- while(true) – break and continue
  - Execute commands in the while loop until it meets break. If it meets continue, it restarts from the beginning
- do – while(condition)
  - First execute command in the do statement and check condition in while to check if it will continue to execute

Loop Example

- while.js

```
var arr1 = [10, 5, 9, -3, 4, 5];
var index = 0;

while (arr1[index] > 0) {
  console.log("Positive value: " + arr1[index++])
}

while (true) {
  var sum_input = parseInt(prompt("2 + 5 = "));
  if (sum_input != 7) {
    console.log("a wrong answer. try again");
    continue;
  }

  console.log("pass add question");

  var mul_input = parseInt(prompt("2 * 5 = "));
  if (mul_input == 10) {
    console.log("pass multiplication question");
    break;
  }
  console.log("failed multiplication question.");
}
```

Loop Example

- do_while.js

```
var answer = parseInt(prompt("5 * 7 = "));
while(answer != 35) {
  answer = parseInt(prompt("5 * 7 = "));
}
console.log("You got a correct answer");

do {
  answer = parseInt(prompt("5 * 7 = "));
} while (answer != 35)
console.log("You got a correct answer with do-while");
~
```

For Loop in Javascript

- for (initialize; condition; update)
  - The initialize code always executes
  - If the condition is not true, the body does not execute
  - After the body execution, update is executed
  - We can omit any of three statements where necessary
  - break and continue in for loop
    - break – always escape the current for loop
    - continue – perform update and check condition then execute

For Loop Example

```
arr1 with while loop: 1
arr1 with while loop: 3
arr1 with while loop: 5
arr1 with while loop: 7
arr1 with while loop: 9
arr1 with for loop: 1
arr1 with for loop: 3
arr1 with for loop: 5
arr1 with for loop: 7
arr1 with for loop: 9
print even number: 2
print even number: 4
print even number: 6
print even number: 8
```

```javascript
var arr1 = [1,3,5,7,9];
var index = 0;

while (arr1[index] != undefined) {
  console.log("arr1 with while loop: " + arr1[index]);
  index++;
}

for(var index = 0; arr1[index] != undefined; index++) {
  console.log("arr1 with for loop: " + arr1[index]);
}

var arr2 = [1,2,3,4,5,6,7,8,9];
for(var index = 0; ; index++) {
  if (arr2[index] == undefined) {
    break;
  }

  if (arr2[index] % 2 != 0) {
    continue;
  } else {
    console.log("print even number: " + arr2[index])
  }
}
```

Iterate For Loop Using in

- Iterate through object keys using in operator
  - for (var key in object)
    - key is the key of the object

```
var input = {name: "kyungyong", work: "KMU", age: 100};

for(var i in input) {
  console.log("key is: " + i + " value is " + input[i]);
}
```

```
key is: name value is kyungyong
key is: work value is KMU
key is: age value is 100
```

Javascript Variable Scope

- A variable declared with var keyword is scoped
  - In a function, if it is declared in a function
  - Global variable if it is declared outside of a function - a function can access it
- A variable without var keyword always becomes a global variable

Variable Scope Example

- var_scope.js

```javascript
var global_var = 123;
console.log(global_var);

function test1() {
  var local_var = 234;
  console.log(global_var + " " + local_var);
  global_var = 456;
}

test1();
console.log(global_var);

function test2() {
    global_val_function = 789;
}

test2();
console.log(global_val_function);

console.log(local_var);
```

```
123
123 234
456
789
Uncaught ReferenceError: local_var is not defined
```

Javascript Prototype

- Every Javascript object inherit properties and methods from a prototype – cf. class inheritance in Java
- A string object inherits all the properties and methods from a String prototype

```
> var str = "sample";
<· undefined
> str.split
    bold
    charAt
    charCodeAt
    codePointAt
    concat
    constructor
    endsWith
    fixed
    fontcolor
    fontsize
    hasOwnProperty
    includes
    indexOf
```

```
propertyIsEnumerable
repeat
replace
search
slice
small
split
startsWith
strike
sub
substr
substring
sup
toLocaleLowerCase
toLocaleString
toLocaleUpperCase
toLowerCase
toString
toUpperCase
trim
trimLeft
trimRight
```

Access Prototype

- Objects have a prototype where it inheritied from
  - Object create from a literal - var str = “sample”;
  - Object created by using new keywork – var d = new Date();
  - The prototype can be accessed Object.prototype

```
> String.prototype
<· ▼ String {length: 0, [[PrimitiveValue]]: ""} i
    ▶ anchor: function anchor()
    ▶ big: function big()
    ▶ blink: function blink()
    ▶ bold: function bold()
    ▶ charAt: function charAt()
    ▶ charCodeAt: function charCodeAt()
    ▶ codePointAt: function codePointAt()
    ▶ concat: function concat()
```

How to Create Prototype

- Using an object constructor function

```
function PrototypeName() {
        // properties
        // methods
}
```

Example of Creating a Prototype

- prototype.js

```
function Person(first, last, gender, country) {
  this.first = first;
  this.last = last;
  this.gender = gender;
  this.country = country;
}
```

```
> var leeky = new Person("kyungyong", "lee", "male", "South Korea");
<· undefined
> leeky
<· Person {first: "kyungyong", last: "lee", gender: "male", country: "South Korea"}
>
```

Reference

- Javascript, the definitive guide
- W3Schools Javascript